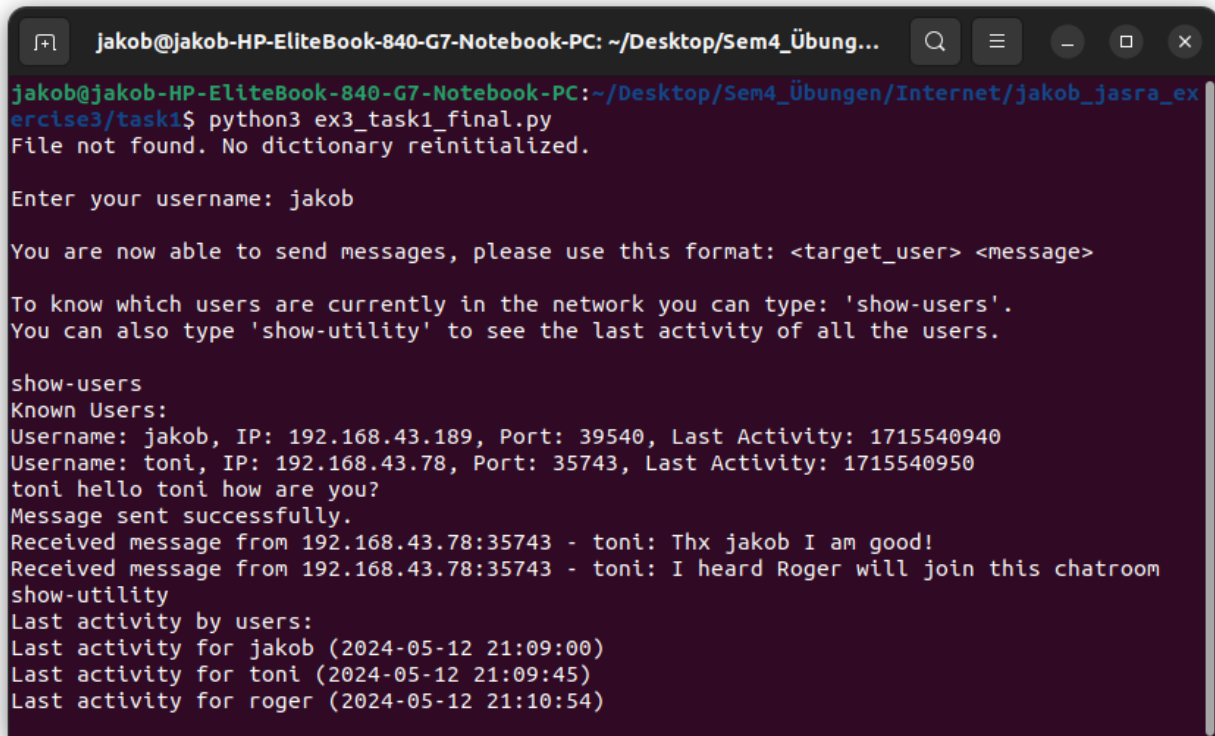


Exercise 3

Task 1



```
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/Sem4_Übung...
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1$ python3 ex3_task1_final.py
File not found. No dictionary reinitialized.

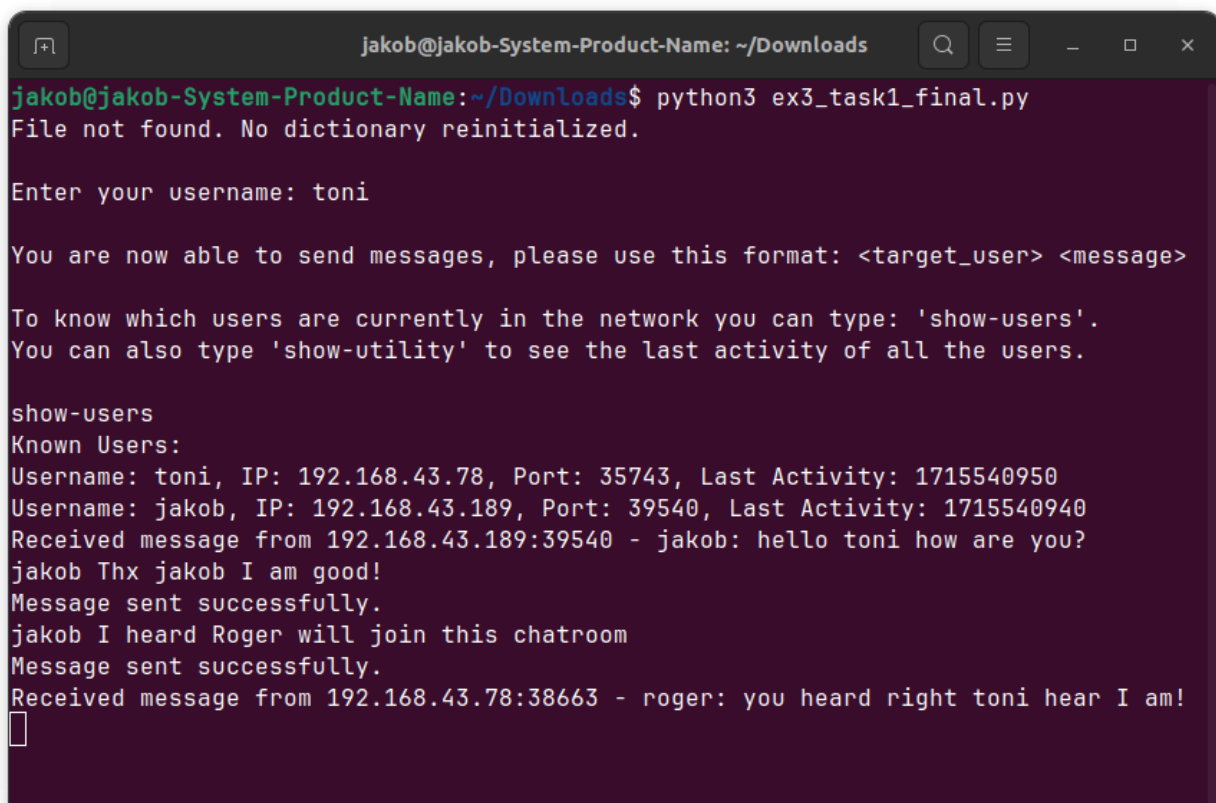
Enter your username: jakob

You are now able to send messages, please use this format: <target_user> <message>

To know which users are currently in the network you can type: 'show-users'.
You can also type 'show-utility' to see the last activity of all the users.

show-users
Known Users:
Username: jakob, IP: 192.168.43.189, Port: 39540, Last Activity: 1715540940
Username: toni, IP: 192.168.43.78, Port: 35743, Last Activity: 1715540950
toni hello toni how are you?
Message sent successfully.
Received message from 192.168.43.78:35743 - toni: Thx jakob I am good!
Received message from 192.168.43.78:35743 - toni: I heard Roger will join this chatroom
show-utility
Last activity by users:
Last activity for jakob (2024-05-12 21:09:00)
Last activity for toni (2024-05-12 21:09:45)
Last activity for roger (2024-05-12 21:10:54)
```

The above picture shows all the messages sent and received for the user jakob. It also shows the use and output of the show-users and show-utility commands (the last activity in show-users is still the unformatted unix time).



```
jakob@jakob-System-Product-Name: ~/Downloads
jakob@jakob-System-Product-Name:~/Downloads$ python3 ex3_task1_final.py
File not found. No dictionary reinitialized.

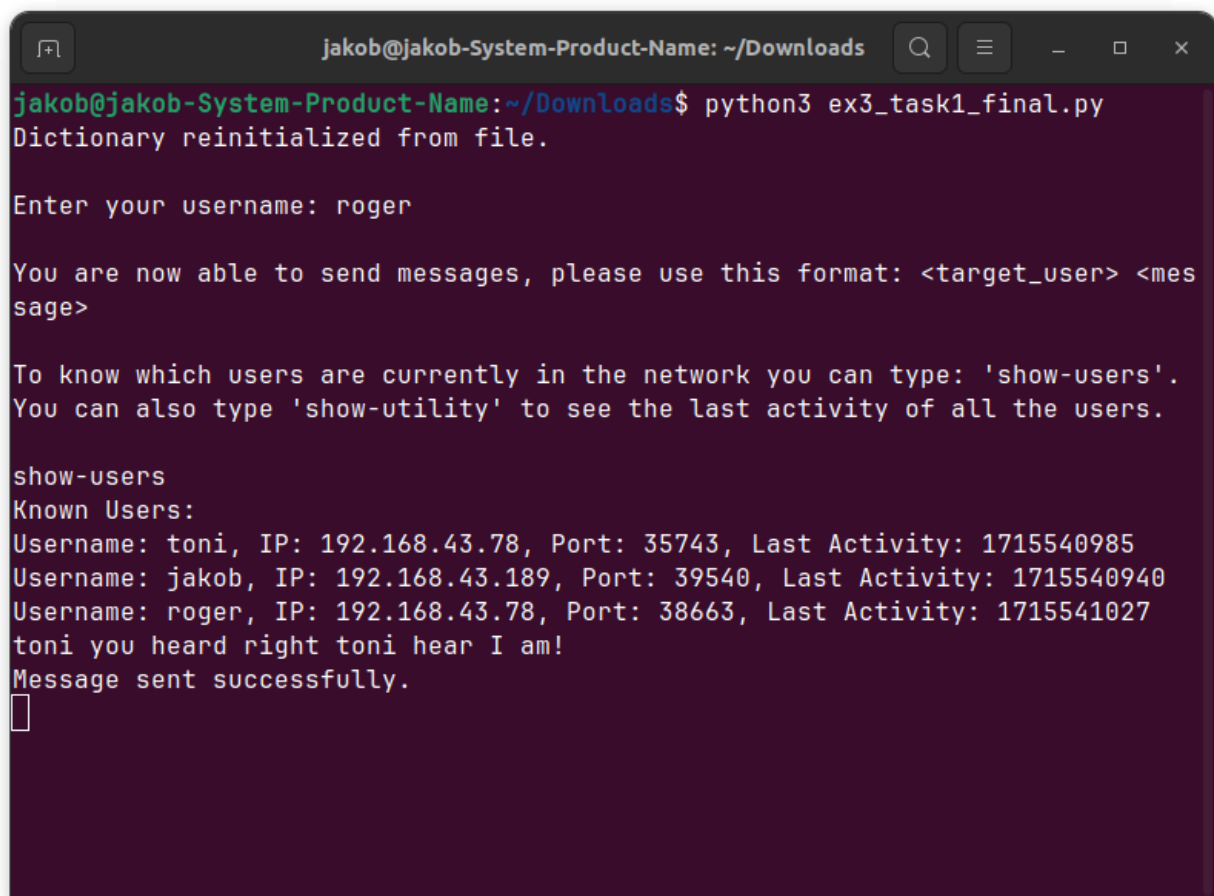
Enter your username: toni

You are now able to send messages, please use this format: <target_user> <message>

To know which users are currently in the network you can type: 'show-users'.
You can also type 'show-utility' to see the last activity of all the users.

show-users
Known Users:
Username: toni, IP: 192.168.43.78, Port: 35743, Last Activity: 1715540950
Username: jakob, IP: 192.168.43.189, Port: 39540, Last Activity: 1715540940
Received message from 192.168.43.189:39540 - jakob: hello toni how are you?
jakob Thx jakob I am good!
Message sent successfully.
jakob I heard Roger will join this chatroom
Message sent successfully.
Received message from 192.168.43.78:38663 - roger: you heard right toni hear I am!
```

Sent and received messages for user toni.



```
jakob@jakob-System-Product-Name: ~/Downloads
jakob@jakob-System-Product-Name:~/Downloads$ python3 ex3_task1_final.py
Dictionary reinitialized from file.

Enter your username: roger

You are now able to send messages, please use this format: <target_user> <message>

To know which users are currently in the network you can type: 'show-users'.
You can also type 'show-utility' to see the last activity of all the users.

show-users
Known Users:
Username: toni, IP: 192.168.43.78, Port: 35743, Last Activity: 1715540985
Username: jakob, IP: 192.168.43.189, Port: 39540, Last Activity: 1715540940
Username: roger, IP: 192.168.43.78, Port: 38663, Last Activity: 1715541027
toni you heard right toni hear I am!
Message sent successfully.
█
```

Sent and received messages for user roger.

Answers to questions for task 1:

a) Describe your state-based CRDT design according to its partial order, merge, and monotonic operations.

I use a dictionary called 'known_users' that has this format:
key:username, value:(ip, port, last_activity)

This dictionary behaves as a grow-only set.

Partial order: $state1 \leq state2$, because state2 will always (by how operations on the dict are done) hold all the keys of state1 and possibly more. For each key in both state1 and state2 of the dict the last-activity entry of the tuple value in state2 will be bigger or equal to the entry in state1.

Merge: A merge happens when the local dictionaries are broadcasted to the and received from the broadcast address (parse_broadcasts method). All the unknown keys with their values from the received state are added to the local state and all the known keys are updated if the received timestamp values are bigger than the ones currently stored in the local state. This behavior follows these least upper bound conditions:

Merge (LUB) operation:if $e \notin \text{keys}(S_i)$ then $S_i[e] =$

$$S_k \leftarrow S_i \sqcup S_j : \text{keys}(S_k) = \text{keys}(S_i) \cup \text{keys}(S_j) \text{ and}$$

$$\text{for each } e \in \text{keys}(S_k) \ S_k[e] = \max(S_i[e], S_j[e])$$

45

Monotonic operations: The behavior is also monotonic like this:

Monotonic Operation(s): $S_{i+1} \leftarrow \text{add}(S_i, e \in \mathcal{E}) : S_{i+1} = S_i \cup \{e\}$

Since keys can only get added and the resulting state is the union of the previous state with the added key (keys can't get removed).

b) Describe how the state of your CRDT is encoded in a single UDP packet with a maximum size of 576 bytes¹ and the associated limitations on the number of participants and the size of participants' user names.

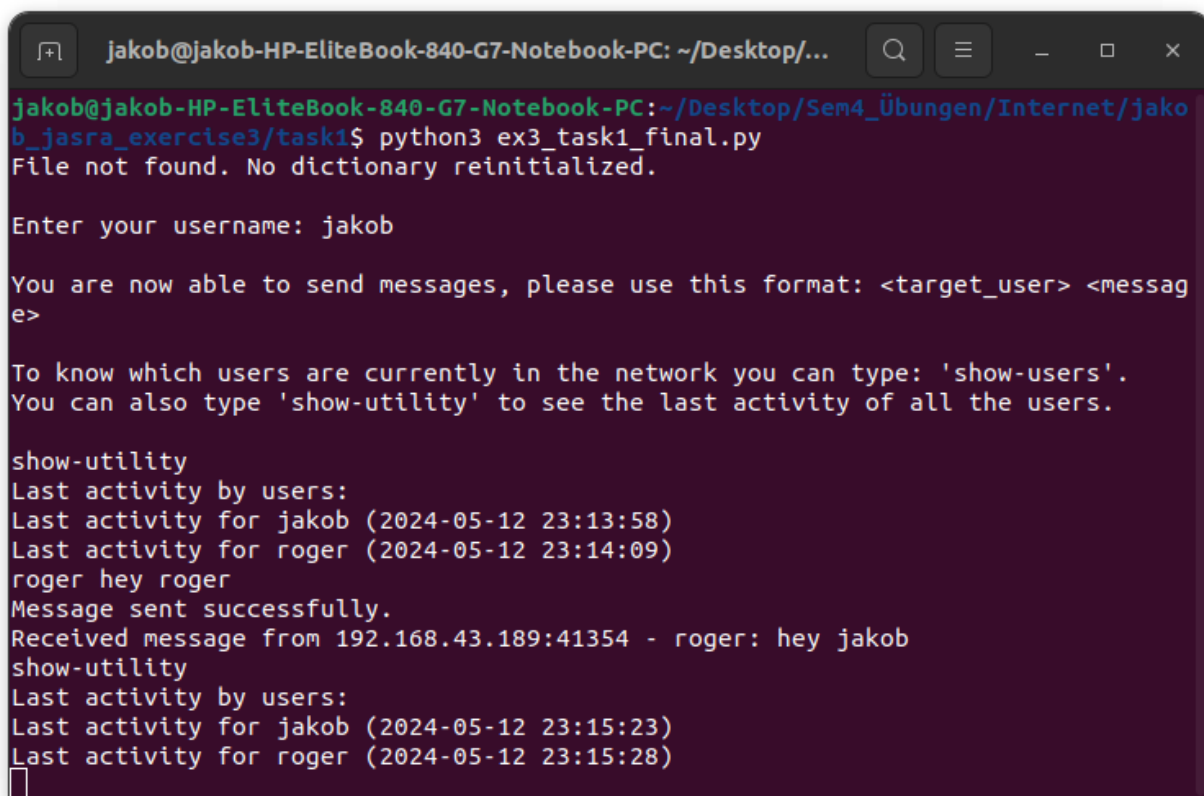
I broadcast the packets like this (broadcast_state() method):

```
state_message = ""
for user, user_data in known_users.items():
    state_message += f"{user}: {user_data[0]}: {user_data[1]}: {user_data[2]}\n"
s.sendto(state_message.encode(), ('<broadcast>', 37020))
#FOR DEBUGGING
```

I can assume that the value tuples each take up IPv4 address = 15 characters + port = 20 characters + time = 30 characters + whitespaces + :s = 34 characters.

If we say the average username has 5 characters + 1 : and 1 whitespace = 7 characters, we can assume that each users key-value pair takes up 34+7=41 characters. Since 1 character is 1 byte in ASCII, each user sends 41 bytes. So we could send information for 576/41=14 about 14 users.

c) Implement your utility and test with at least 2 machines to verify that all replicas are periodically consistent, i.e. they periodically show the same last recorded activity for all participants, even if one of the replicas is temporarily offline.



```
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/...
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1$ python3 ex3_task1_final.py
File not found. No dictionary reinitialized.

Enter your username: jakob

You are now able to send messages, please use this format: <target_user> <message>

To know which users are currently in the network you can type: 'show-users'.
You can also type 'show-utility' to see the last activity of all the users.

show-utility
Last activity by users:
Last activity for jakob (2024-05-12 23:13:58)
Last activity for roger (2024-05-12 23:14:09)
roger hey roger
Message sent successfully.
Received message from 192.168.43.189:41354 - roger: hey jakob
show-utility
Last activity by users:
Last activity for jakob (2024-05-12 23:15:23)
Last activity for roger (2024-05-12 23:15:28)
█
```

Jakobs view

```
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/...  
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1$ python3 ex3_task1_final.py  
Dictionary reinitialized from file.  
  
Enter your username: roger  
  
You are now able to send messages, please use this format: <target_user> <message>  
  
To know which users are currently in the network you can type: 'show-users'.  
You can also type 'show-utility' to see the last activity of all the users.  
  
show-utility  
Last activity by users:  
Last activity for jakob (2024-05-12 23:13:58)  
Last activity for roger (2024-05-12 23:14:09)  
^C  
Traceback (most recent call last):  
  File "/home/jakob/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1/ex3_task1_final.py", line 264, in <module>  
    main()  
  File "/home/jakob/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1/ex3_task1_final.py", line 261, in main  
    time.sleep(1)  
KeyboardInterrupt
```

Roger chats a bit, then is interrupted.

```
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/...  
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task1$ python3 ex3_task1_final.py  
Dictionary reinitialized from file.  
  
Enter your username: roger  
  
You are now able to send messages, please use this format: <target_user> <message>  
  
To know which users are currently in the network you can type: 'show-users'.  
You can also type 'show-utility' to see the last activity of all the users.  
  
Received message from 192.168.43.189:45198 - jakob: hey roger  
jakob hey jakob  
Message sent successfully.  
show-utility  
Last activity by users:  
Last activity for jakob (2024-05-12 23:15:23)  
Last activity for roger (2024-05-12 23:15:28)  
█
```

Roger comes online again and if you look at jakobs screen above and then rogers right here (I typed show-utility into both terminals), they display the correct last activity times for both users. Roger as seen in the middle terminal went offline between some exchanged messages.

Task2

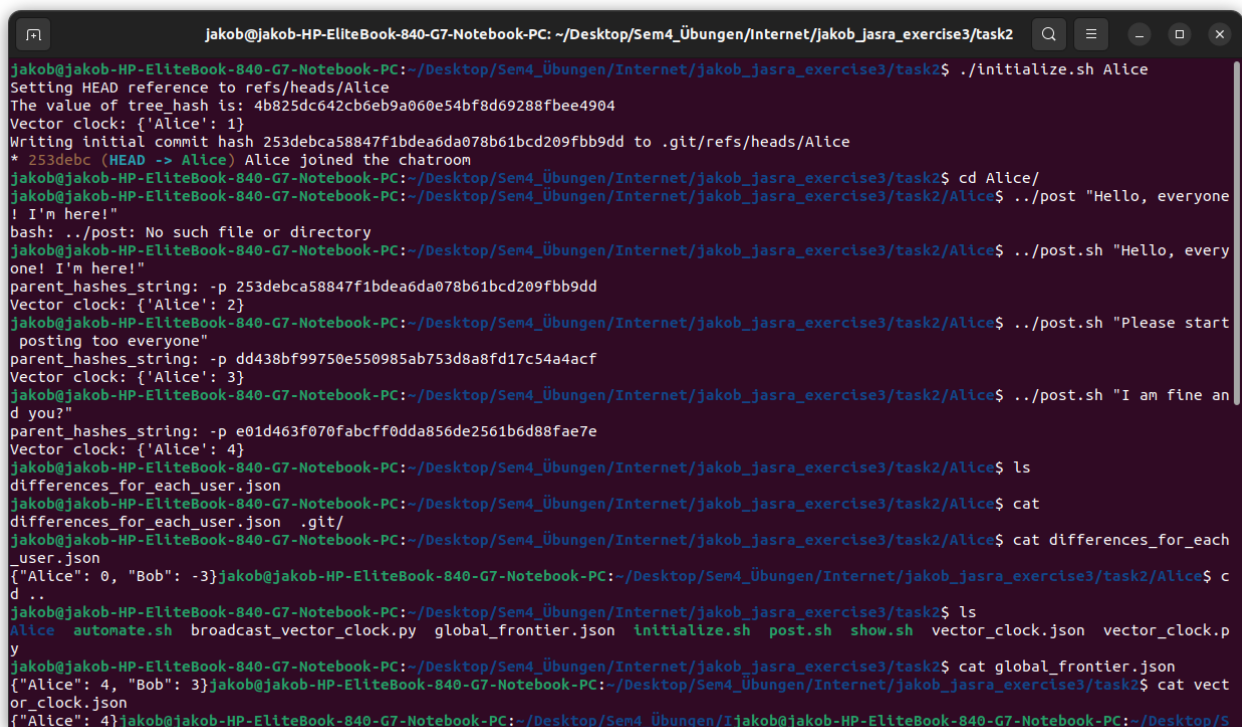
“””””

To correctly update the vector clock, in task2, the post function from exercise 2 has to increment the vector clock (the process is that current posting user).

Vector clock is implemented with dictionary

“”””” (→ Deal with this block of text)

First I initialize 2 repositories Alice & Bob on different machines, then I start broadcasting (in a different terminal, since this is a running program, note that you have to call this script out of the relevant repository like in the below screenshots) and then I start posting messages.



```

jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2$ ./initialize.sh Alice
Setting HEAD reference to refs/heads/Alice
The value of tree_hash is: 4b825dc642cb6eb9a060e54bf8d69288fbee4904
Vector clock: {'Alice': 1}
Writing initial commit hash 253debca58847f1bdea6da078b61bcd209fbb9dd to .git/refs/heads/Alice
* 253debc (HEAD -> Alice) Alice joined the chatroom
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2$ cd Alice/
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ ../post "Hello, everyone
! I'm here!"
bash: ../post: No such file or directory
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ ../post.sh "Hello, every
one! I'm here!"
parent_hashes_string: -p 253debca58847f1bdea6da078b61bcd209fbb9dd
Vector clock: {'Alice': 2}
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ ../post.sh "Please start
posting too everyone"
parent_hashes_string: -p dd438bf99750e550985ab753d8a8fd17c54a4acf
Vector clock: {'Alice': 3}
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ ../post.sh "I am fine an
d you?"
parent_hashes_string: -p e01d463f070fabcf0dda856de2561b6d88fae7e
Vector clock: {'Alice': 4}
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ ls
differences_for_each_user.json
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ cat
differences_for_each_user.json .git/
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ cat differences_for_each
_user.json
{"Alice": 0, "Bob": -3}jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ c
d ..
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2$ ls
Alice automate.sh broadcast_vector_clock.py global_frontier.json initialize.sh post.sh show.sh vector_clock.json vector_clock.p
y
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2$ cat global_frontier.json
{"Alice": 4, "Bob": 3}jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2$ cat vect
or_clock.json
{"Alice": 4}jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/S

```

In the cat commands (in the above picture) you can see that the global_frontiers, the ‘differences_for_each’ (signifying how ahead / behind the local branches are) and the (local) vector_clock are all output correctly. Alice has 4 commits is 3 behind on Bobs ‘branch’ and the global_frontiers shows the correct ‘global’ vector_clock.


```
jakob@jakob-System-Product-Name: ~/Downloads/Bob
jakob@jakob-System-Product-Name:~/Downloads$ ./initialize.sh Bob
Setting HEAD reference to refs/heads/Bob
The value of tree_hash is: 4b825dc642cb6eb9a060e54bf8d69288fbee4904
Vector clock: {'Bob': 1}
Writing initial commit hash 9947ac538123c920c058eb24a97db0c7c2f51f55 to .git/refs/heads/Bob
* 9947ac5 (HEAD -> Bob) Bob joined the chatroom
jakob@jakob-System-Product-Name:~/Downloads$ cd Bob/
jakob@jakob-System-Product-Name:~/Downloads/Bob$ ../post.sh "Ok I'll start chatting, how are you Alice?"
parent_hashes_string: -p 9947ac538123c920c058eb24a97db0c7c2f51f55
Vector clock: {'Bob': 2}
jakob@jakob-System-Product-Name:~/Downloads/Bob$ ../post.sh "I'm good thx"
parent_hashes_string: -p 366b23d760212ae5183d47cf91d3e4748ea31789
Vector clock: {'Bob': 3}
jakob@jakob-System-Product-Name:~/Downloads/Bob$
```

```
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC: ~/Desktop/Sem4_Ü...
jakob@jakob-HP-EliteBook-840-G7-Notebook-PC:~/Desktop/Sem4_Übungen/Internet/jakob_jasra_exercise3/task2/Alice$ python3 ../broadcast_vector_clock.py
```

(This has to be done on both sides, so from Alice and Bob's repositories)

For the logic of the local_frontier broadcasting, the global_frontier and differences_between_users computation and the tracking of other replicas, I think its easier to look at the broadcast_vector_clock.py.

a) Concerning how the local frontier is computed from the local message graph: Whenever commits are created in my bash scripts (initialize.sh and post.sh from ex2) I call the vector_clock.py script (from my bash script) which updates a persistent (local) vector_clock (which is stored as dict in a .json file, the vector clock is implemented with a dict). (global_frontier and differences_between_users dicts are also stored in jsons).

Partial order: There is partial order with regards to the global_frontier that is calculated across participating users, each key in any local vector clocks is in the global_frontier vector clock and the values (messages posted counters for the different users as key:user, value:messages_posted dict) in the global_frontier vector clock are always the highest known across all the locally existing message graphs.

Merge: A merge happens when the local vector_clocks are broadcasted to the and received from the broadcast address (parse_broadcasts method). As in task1 the unknown keys with their values from the received state are added to the local state and all the known keys are updated if the received timestamp values are bigger than the ones currently stored in the local state. This behavior follows the least upper bound conditions like in task1.

Why adding a new message to the local message graph is a monotonic operation: The counters in the global_frontier vector clock respects these conditions:

Monotonic Operation(s): $x_{i+1} \leftarrow inc(x_i, y \geq 0) : x_{i+1} = x_i + y$

Def (least upper bound): an operation $x_k \leftarrow op(x_i, x_j)$ computes the least upper bound of x_i, x_j **iff** for all possible values of x_i, x_j :

1. $x_k \supseteq x_i$
2. $x_k \supseteq x_j$
3. there does not exist another $x_{k'}$ such that $x_{k'} \supseteq x_i$ and $x_{k'} \supseteq x_j$ and $x_{k'} \subset x_k$.

If x_i and x_j are added both are smaller or equal to the result (local vector clock values through broadcast getting added to the global_frontier), since not anymore than the amount that is bigger is added (just the max is chosen) condition 3 also holds.

b) Similar considerations as in task 1 apply, since I allow to receive 1024 bytes here I assume at least $(1024/576)*14$ users data could get sent here, but since the sent data is only about 7 bytes per user this could be multiplied by $41/7$ too and give about 145 users who could participate. With only 576 bytes about 82. $(14*(41/7))$.

c) Code from `def update_local_references(received_user, received_vector_clock):` to main method in broadcast_vector_clock.py.